# CPC - Programming Exercise

Michael Sell

12/15/2024

## Part 1:

*Code a function (subroutine) that generates the BA model for an arbitrary value of N and m. To store the network structure, it is useful to use and adjacency list, that is, a 1 list of lists that associates to each node a list with its nearest neighbors. Instead of a list of lists, a dictionary can be used, that for each key, corresponding to the index of each node, assigns as value the list of its nearest neighbors. Lists are useful since they can grow in the process of generating the BA network. For the generating process, one needs to select old nodes with probability Eq. (1) to connect the newly added nodes. Sampling from such changing distribution can be done using an auxiliary list to select the vertices to which new edges are attached. Add to this array the number i, every time you add a new connection to vertex i, and m times the number j whenever you add the new vertex j. Any number i will appear in this way ki times. Selecting uniformly at random an element in this auxiliary array will give you the number i with probability proportional to ki. Do not forget the initial nucleus of the network. You can use the random number generator provided by Python (the high-quality Mersenne-Twister)*

In [83]:
```
import matplotlib.pyplot as plt
import numpy as np
from collections import Counter
import random
from scipy.optimize import curve_fit
```

### FUNCTION: `Generate BA Network`

In [84]:
```
def generate_BA(N,m):

    # Initialize Library
    Network = {node: [] for node in range(N)}



    # Initial nucleus size
    n0 = m + 1

    # Connect every node in the nucleus to each other
    for node1 in range(n0):
            for node2 in range(node1 + 1 , n0):  # More efficient than adding (if node1 != nod
                Network[node1].append(node2)
                Network[node2].append(node1)

    # Debug
    # print(Network)

    # Initialize Aux. list
    aux_list = []
```

```
    # Initialize initial nucleus nodes
    for node in range(n0):
        for i in range(len(Network[node])):
            aux_list.append(node)

    # Debug
    # print(aux_list)


    # Add new nodes from n0 to N (where j is the new node)
    for j in range(n0, N):

        connected_nodes = []
        while len(connected_nodes) < m:
            # Randomly select attachment node
            the_chosen_node = random.choice(aux_list)
            if the_chosen_node not in connected_nodes:
                connected_nodes.append(the_chosen_node)

                # Append 'the_chosen_node' at the index 'new_node' in Network
                # Connect the two using the same method as the initial nucleus
                Network[j].append(the_chosen_node)
                Network[the_chosen_node].append(j)


        # Add number i to each connection made
        for i in connected_nodes:
            aux_list.append(i)

        # Add the new node, j * m for each new vertex i
        aux_list.extend([j] * m)

    return Network
```

## FUNCTION: `Degree Distribution`

We create another function to plot the degree distribution that returns `k_list` which is a list of the degrees with the index being the node.

In [85]:
```
def create_degree_list(Network, m):
    k_list = []
    mf_approx_list = []

    for node in Network:
        k = len(Network[node])  # Degree of the node
        k_list.append(k)

        # Correct Mean Field Approximation
        mf_approx = (2 * (m**2)) / (k**3) if k > 0 else 0  # Avoid division by zero
        mf_approx_list.append(mf_approx)

    return k_list
```

## FUNCTION: `Plot Probabilities`

Plot Equations 3 and 4 Against Degree Distribution

In [86]:
```
def plot_probabilities(k_list, N, m):
```

```
    # Incorporate Eq. (3) and (4)

    # To handle floats in arrays
    k_list = np.array(k_list)
    degree = np.linspace(k_list.min(), k_list.max(), num=100)

    # Compute P_inf for each degree k in degree
    P_inf = []
    P_mf = []
    P_N = []
    for k in degree:

        # Compute P_inf (Equation 3)
        P_inf_i = (2 * m * (m + 1)) / (k * (k + 1) * (k + 2))
        P_inf.append(P_inf_i)

        # Compute P_mf (Equation 2)
        P_mf_i = (2 * (m**2)) / (k**3) if k > 0 else 0
        P_mf.append(P_mf_i)

        # # Compute P_N (Equation 4)
        # P_N_i = P_inf_i * (k / np.sqrt(N))
        # P_N.append(P_N_i)

    # P_inf
    plt.figure()
    plt.plot(degree, P_inf, label="P_inf(k)", linewidth=3, color='skyblue')
    plt.xscale('log')
    plt.yscale('log')
    plt.xlabel("k")
    plt.ylabel("P_inf")
    plt.grid(True)
    plt.legend()

    # P_N
    plt.plot(degree, P_mf, label="P_mf(k)", linewidth=3, color='blue')
    plt.xscale('log')
    plt.yscale('log')
    plt.xlabel("k")
    plt.ylabel("P_mf")
    plt.grid(True)
    plt.legend()

    return P_inf
```

## FUNCTION: `Plot Averaged Probablilites`

This is to account for all the averaged probabilities. Returns P_inf and P_N

In [87]:
```python
def plot_averaged_probabilities(hist_avg, bins, N, m):

    # Degree array
    degree = np.array(bins[:-1])

    # Find the first non-zero bin in the hist_avg (Un-comment to optimize graph limits)
    first_non_zero_index = np.argmax(hist_avg > 0)

    # Find first non-zero degree to start Probability calculation at
    trimmed_degree = degree[first_non_zero_index:]
    # trimmed_hist_avg = hist_avg[first_non_zero_index:]

    # Compute trimmed P_inf and P_N
    P_inf = []
    P_mf = []
    P_N = []

    for k in trimmed_degree:
        if k > 0:  # Avoid division by zero for k = 0
            # Compute P_inf (Equation 3)
            P_inf_i = (2 * m * (m + 1)) / (k * (k + 1) * (k + 2))
            P_inf.append(P_inf_i)

            # Compute P_mf (Equation 2)
            P_mf_i = (2 * (m**2)) / (k**3) if k > 0 else 0
            P_mf.append(P_mf_i)

            # # Compute P_N (Equation 4)
            # P_N_i = P_inf_i * (k / np.sqrt(N))
            # P_N.append(P_N_i)
        else:
            P_inf.append(0)
            P_N.append(0)


    # Plot P_inf and P_N
    plt.figure()
    plt.plot(trimmed_degree, P_inf, label="P_inf(k)", linewidth=3, color='skyblue')
    plt.plot(trimmed_degree, P_mf, label="P_mf(k)", linewidth=3, color='blue')
    plt.xscale('log')
    plt.yscale('log')

    plt.xlabel("k (Degree)")
    plt.ylabel("Probability")
    plt.legend()
    plt.grid(True)

    ## Optional Axis Limits
    # plt.xlim(trimmed_degree[0], trimmed_degree[-1])
```

## FUNCTION: `Compute P_inf`

In [88]:
```python
def compute_p_inf(hist_avg, bins, m):

    degree = np.array(bins[:-1])

    # Compute Pinfvals
    P_inf_vals = []
```

```
    for k in degree:
        if k > 0:  # Avoid division by zero for k = 0
            # Compute P_inf (Equation 3)
            P_inf_i = (2 * m * (m + 1)) / (k * (k + 1) * (k + 2))
            P_inf_vals.append(P_inf_i)
        else:
            P_inf_vals.append(0)

    return np.array(P_inf_vals)
```

## FUNCTION: `Plot Degree Distribution`

In [89]:
```
def plot_degree_distribution(k_list, N, m):


    # Convert to array
    k_list = np.array(k_list)

    # Compute unique degrees frequencies
    unique_degrees, counts = np.unique(k_list, return_counts=True)

    # Normalize frequencies to obtain probabilities
    probabilities = counts / len(k_list)

    # Plot the degree distribution as discrete points
    plt.plot(unique_degrees, probabilities, 'o', markersize=3, label='Degree Distribution', co

    # Add labels, title, and legend
    plt.xlabel('Degree (k)')
    plt.ylabel('Probability')
    plt.xscale('log')
    plt.yscale('log')
    plt.title(f'Degree Distribution with Infinite Network Probability\nNetwork Size: {N}')
    plt.legend()
    plt.grid(True)
    plt.show()
```

## FUNCTION: `Plot Averaged Degree Distribution`

Separate Function from simply plotting the k_list as from above

In [90]:
```
def plot_averaged_degree_distribution(hist_avg, N, m):

    hist_avg = np.array(hist_avg)

    degrees = np.arange(len(hist_avg))

    hist_avg_normalized = hist_avg / np.sum(hist_avg)

    lowest_value = degrees[np.nonzero(hist_avg_normalized)[0][0]]


    # Plot averaged degree distribution as discrete points
    plt.plot(degrees, hist_avg_normalized, 'o', markersize=4, label='Averaged Degree Distribut
    plt.xlim(left=lowest_value - 1)
    plt.xlabel('Degree (k)')
    plt.ylabel('Probability')
    plt.xscale('log')
    plt.yscale('log')
```

```
    plt.title(f'Averaged Degree Distribution with Infinite Network Probability\nNetwork Size:
    plt.legend()
    plt.grid(True)
    plt.show()
```

## FUNCTION: `Plot Finite Size Correction`

In [91]:
```
def plot_finite_size_correction(hist_avg, bins, N_vals, m, M):
    plt.figure()
    for N in N_vals:
        # Initialize sum of histograms for averaging
        hist_sum = None

        # Initialize Max Degree Value
        max_degree = 0

        for network in range(M):
            Network_i = generate_BA(N, m)
            k_list = create_degree_list(Network_i, m)

            # Keep track of max_degree encountered
            max_degree = max(max_degree, max(k_list))

        # Define consitent bins for np.histogram()
        bins = range(0, max_degree + 2)

        for network in range(M):
            Network_i = generate_BA(N, m)
            k_list = create_degree_list(Network_i, m)

            hist, _ = np.histogram(k_list, bins=bins) # Need to use consistent bins here

            if hist_sum is None:
                hist_sum = hist
            else:
                hist_sum += hist

        # Normalization
        hist_avg = hist_sum / M
        hist_avg = hist_avg / np.sum(hist_avg)

        # Degree
        degree = np.array(bins[:-1])

        # Compute the theoretical P_inf values for this network size
        P_inf_vals = compute_p_inf(hist_avg,bins,m) # Outputs an array of P_inf (theoretical v

        correction = hist_avg / P_inf_vals  # Finite size correction

        # Plot the correction
        plt.plot(degree, correction, label=f'N = {N}', linewidth=2)

        # Enable grid lines
        plt.grid(which='major', linestyle='-', linewidth=0.5, color='black')  # Major grid
        plt.grid(which='minor', linestyle='--', linewidth=0.5, color='gray')  # Minor grid

        # Turn on minor ticks
        plt.minorticks_on()

        # Labels
        plt.xlabel('Degree k')
```

```
        plt.ylabel(r'$P_{\text{est}}(k) / P_{\infty}(k)$')
        plt.xscale('log')
        plt.yscale('log')
        # plt.grid(True)
        plt.legend()
        plt.title('Finite Size Corrections: Pest(k) / P∞(k)')

    plt.show()
```

## FUNCTION: `Compute w(x) Function`

In [92]:

```
# Repeat most of the previous function but solving for w(x)
def plot_Wx(hist_avg, bins, N_vals, m, M):
    plt.figure()
    all_x = [] # relationship x = k/sqrt(N)
    all_w = [] # function w(x)

    for N in N_vals:
        # Initialize sum of histograms for averaging
        hist_sum = None

        # Initialize Max Degree Value
        max_degree = 0

        for network in range(M):
            Network_i = generate_BA(N, m)
            k_list = create_degree_list(Network_i, m)

            # Keep track of max_degree encountered
            max_degree = max(max_degree, max(k_list))

        # Define consitent bins for np.histogram()
        bins = range(0, max_degree + 2)

        for network in range(M):
            Network_i = generate_BA(N, m)
            k_list = create_degree_list(Network_i, m)

            hist, _ = np.histogram(k_list, bins=bins) # Need to use consistent bins here

            if hist_sum is None:
                hist_sum = hist
            else:
                hist_sum += hist

        # Normalization
        hist_avg = hist_sum / M
        hist_avg = hist_avg / np.sum(hist_avg)

        # Degree
        degree = np.array(bins[:-1])

        # Compute the theoretical P_inf values for this network size
        P_inf_vals = compute_p_inf(hist_avg,bins,m) # Outputs an array of P_inf (theoretical v

        correction = hist_avg / P_inf_vals  # Finite size correction

        # Compute x
        x = degree / np.sqrt(N)
        all_x.extend(x)
        all_w.extend(correction)
```

```
    all_x = np.array(all_x)
    all_w = np.array(all_w)

    # Plot w(x)
    plt.scatter(all_x, all_w, alpha=0.4, label='w(x)')


    # Enable grid lines
    plt.grid(which='major', linestyle='-', linewidth=0.5, color='black')  # Major grid
    plt.grid(which='minor', linestyle='--', linewidth=0.5, color='gray')  # Minor grid

    # Turn on minor ticks
    plt.minorticks_on()

    # Labels
    plt.xlabel(r'$x = \frac{k}{\sqrt{N}}$')
    plt.ylabel(r'$w(x)$')
    plt.xscale('log')
    plt.yscale('log')
    plt.grid(True)
    plt.title('Universal Function w(x)')
    plt.legend()

    plt.show()
```

## FUNCTION: `Compute Average Clustering Coefficient`

In [93]:

```
def compute_avg_cluster_coef(N_vals,M,m):

    # Compute average cluster coefficient
    avg_cc = []

    for N in N_vals:
        for _ in range(M):
            Network_i= generate_BA(N,m)
            k_list = create_degree_list(Network_i, m)
            cc_sum = 0
            for node in Network_i:
                k = k_list[node]
                connections = Network_i[node] # connections between the node itself and others
                if k < 2:
                    c_i = 0 # k has to be bigger than 2 to form a connection
                else:
                    e_i = 0
                    for i in connections:
                        for j in connections:
                            if i in Network_i[j]:
                                e_i += 1   # Plus one edge
                    c_i = (2 * e_i) / (k * (k - 1))
                cc_sum += c_i

            avg_cc_i = (1 / N) * cc_sum
        avg_cc.append(avg_cc_i / M)

    # Inverse N plot for comparison
    x = np.linspace(min(N_vals), max(N_vals), 500 )
    y = 1 / x

    # Plotting
    plt.figure(figsize=(8, 6))
```

```
    plt.plot(N_vals, avg_cc, marker='o', linestyle='-', color='r', label="Average Clustering C
    plt.plot(x, y, linestyle='-', color='gray', label='1/N Power Law')
    plt.xlabel("Network Size (N)", fontsize=12)
    plt.ylabel("Average Clustering Coefficient", fontsize=12)
    plt.title("Average Clustering Coefficient vs Network Size (N)", fontsize=14)
    plt.grid(True)
    plt.legend()
    plt.tight_layout()

    # Log-Log Plot
    plt.figure(figsize=(8, 6))
    plt.plot(N_vals, avg_cc, marker='o', linestyle='-', color='r', label="Average Clustering C
    plt.plot(x, y, linestyle='-', color='gray', label='1/N Power Law')
    plt.xlabel("Network Size (N)", fontsize=12)
    plt.ylabel("Average Clustering Coefficient", fontsize=12)
    plt.title("Average Clustering Coefficient vs Network Size (N) LOG-LOG Scale", fontsize=14)
    plt.grid(True)
    plt.xscale('log')
    plt.yscale('log')
    plt.legend()
    plt.tight_layout()
    plt.show()

    return avg_cc
```

# Main Code

---

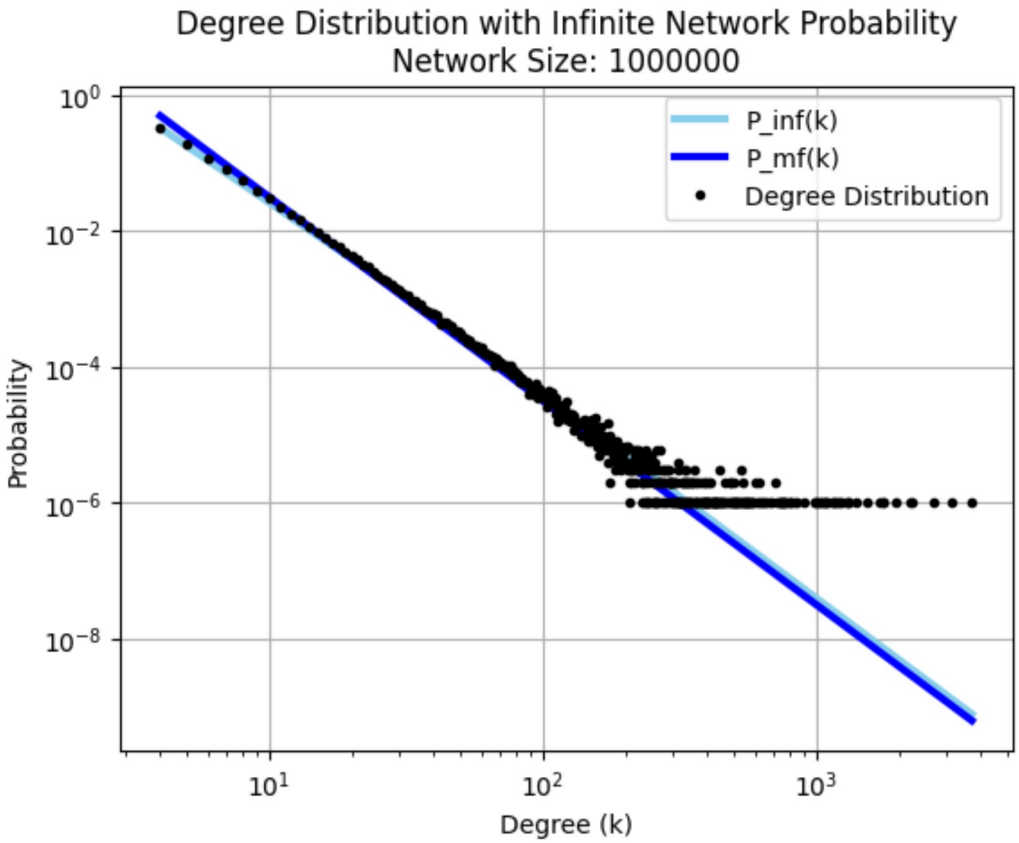
# Part 2:

*For m = 4, generate one BA network of large size, N = 106 if possible or larger, and compute its degree distribution. Make a plot showing that the prediction Eq. (3) is better than the approximation Eq. (4), specially for small values of the degree k.*

- Eq. (3) : $P_\infty(k) = \frac{2m(m+1)}{k(k+1)(k+2)}$
- Eq.(4) : $P_N(k) = P_\infty(k) w(\frac{k}{\sqrt{(N)}})$

From the plot seen below, we can see that the P_inf(k) plot (light blue) corresponds better to the degree distribution than the P_mf(k) (dark blue), especially around low values of degree, k.

In [94]:
```
m = 4
N = 10**6

np.random.seed(9)

# Compute Network for N = 10^6 nodes
Network = generate_BA(N, m)
k_list = create_degree_list(Network, m)
plot_probabilities(k_list, N, m)
plot_degree_distribution(k_list, N, m)
```




# Part 3:

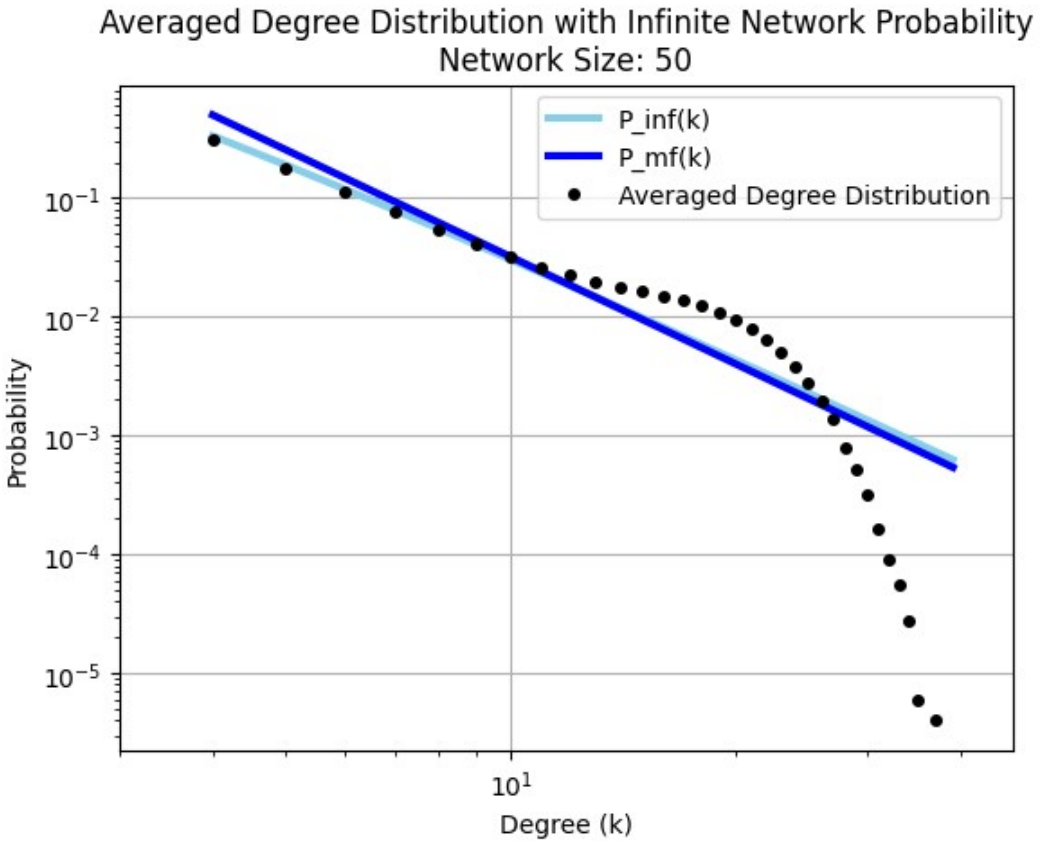
*For m = 4, compute the average degree distribution PN (k) for BA networks with size N = 50, 100, 200 and 500. Perform averages over a number of M = 10000 different networks. For the averages, make a histogram accumulated the degrees of all the networks generates and apply normalization at the end. Check the finite size corrections in Eq. (4) by plotting, for the different values of N the Pest(k)/P∞(k) as a function of k, where Pest(k) is the average degree distribution that you have estimated numerically*

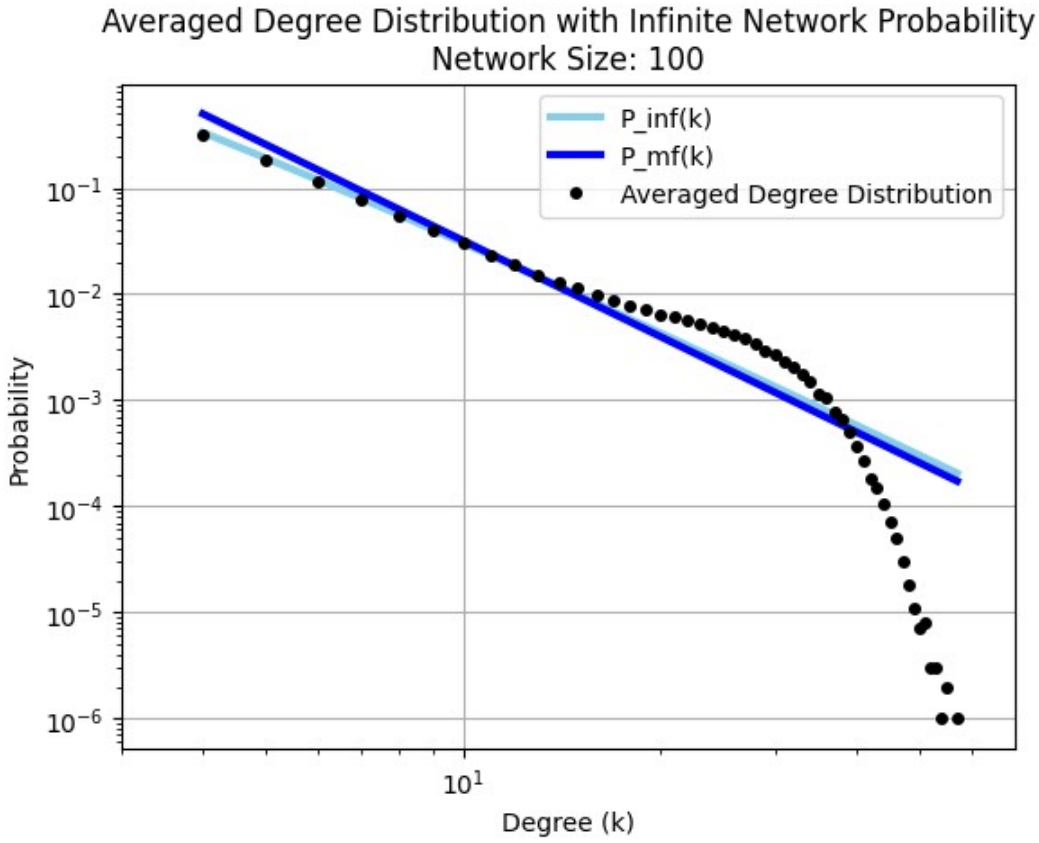
## Histogram Bin Averaging Method:

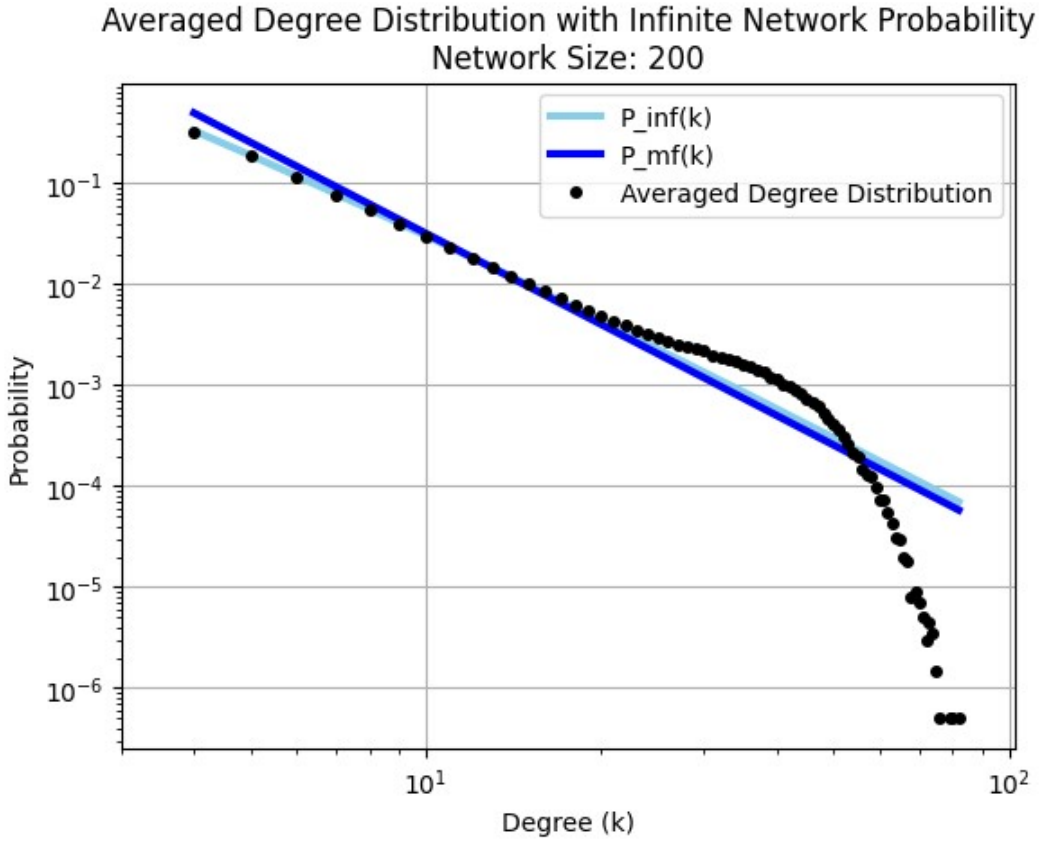
In this method, we can take the summation of the individual bins after creating histograms for each network. This gives us an accurate summation of the degrees across M networks.

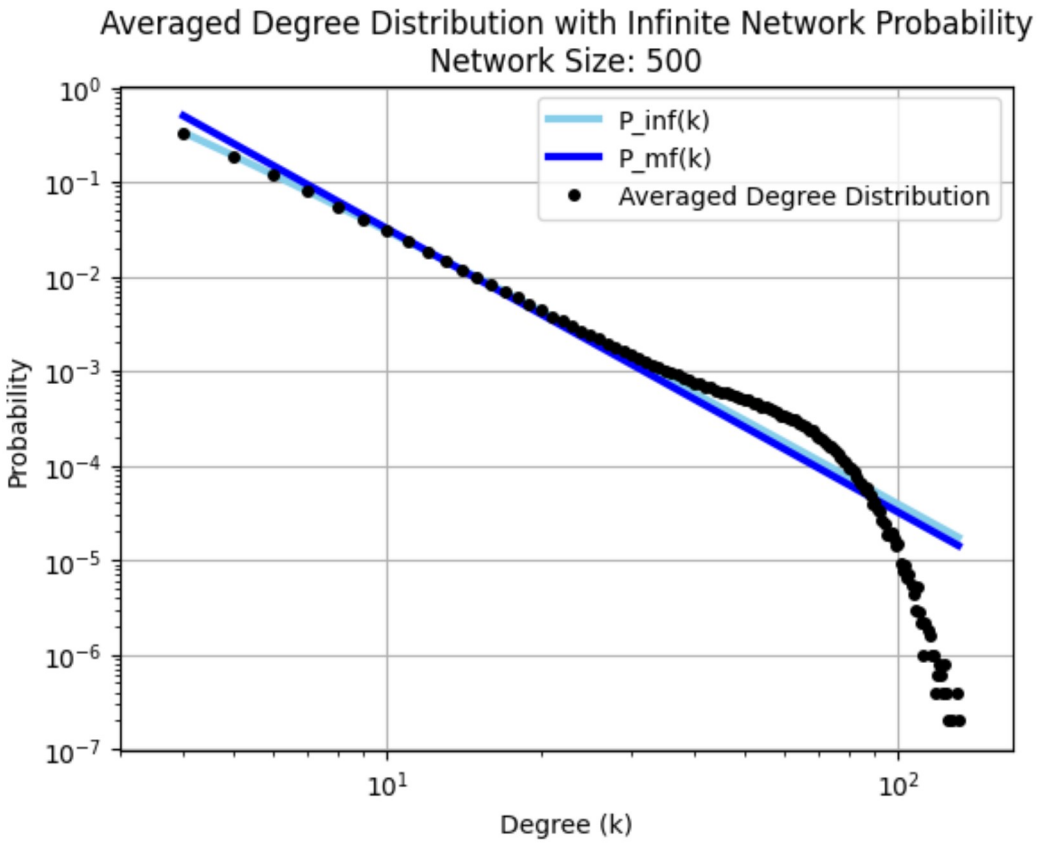
In [95]:
```
# For M different Networks
M = 10000
N_vals = [50, 100, 200, 500]

for N in N_vals:

    hist_sum = None

    # Initialize Max Degree Value
    max_degree = 0

    for network in range(M):
        Network_i = generate_BA(N, m)
        k_list = create_degree_list(Network_i, m)
```

```
        # Keep track of max_degree encountered
        max_degree = max(max_degree, max(k_list))

    # Define consitent bins for np.histogram()
    bins = range(0, max_degree + 2)

    for network in range(M):
        Network_i = generate_BA(N, m)
        k_list = create_degree_list(Network_i, m)

        hist, _ = np.histogram(k_list, bins=bins) # Need to use consistent bins here

        if hist_sum is None:
            hist_sum = hist
        else:
            hist_sum += hist

    # Normalization
    hist_avg = hist_sum / M
    hist_avg = hist_avg / np.sum(hist_avg)

    # Plot the histograms
    plot_averaged_probabilities(hist_avg, bins, N, m)
    plot_averaged_degree_distribution(hist_avg, N, m)
```




Averaged Degree Distribution with Infinite Network Probability
Network Size: 100
P_inf(k)
P_mf(k)
Averaged Degree Distribution
Probability
Degree (k)

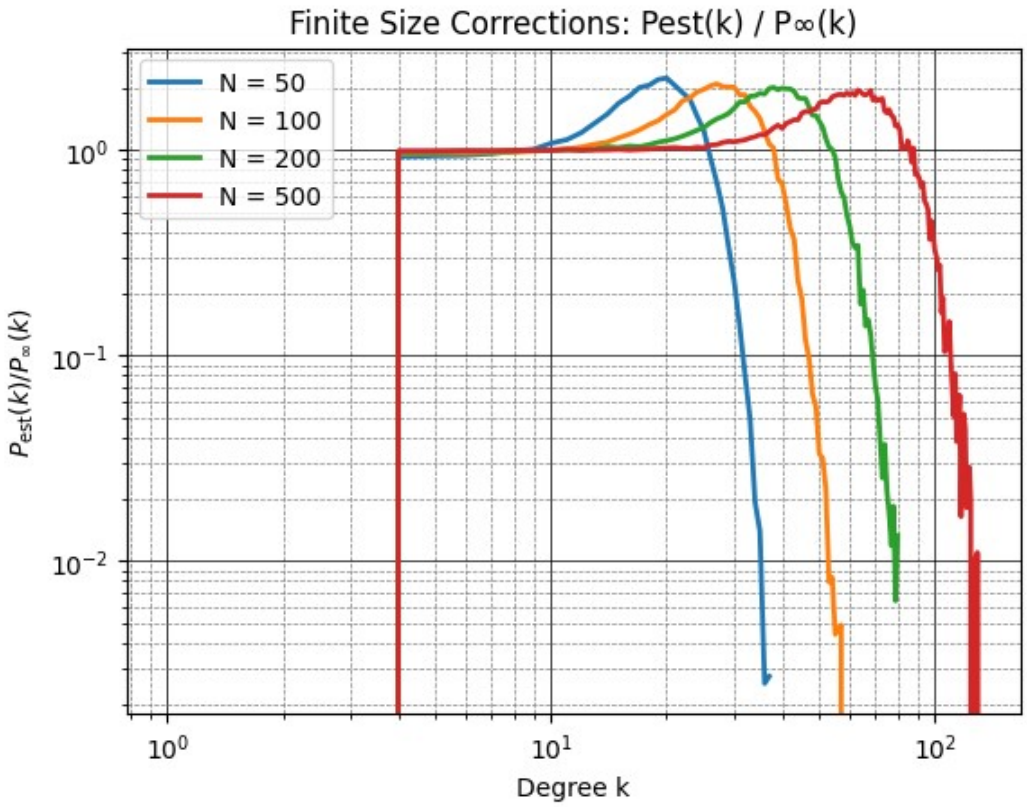

Averaged Degree Distribution with Infinite Network Probability
Network Size: 200
P_inf(k)
P_mf(k)
Averaged Degree Distribution
Probability
Degree (k)



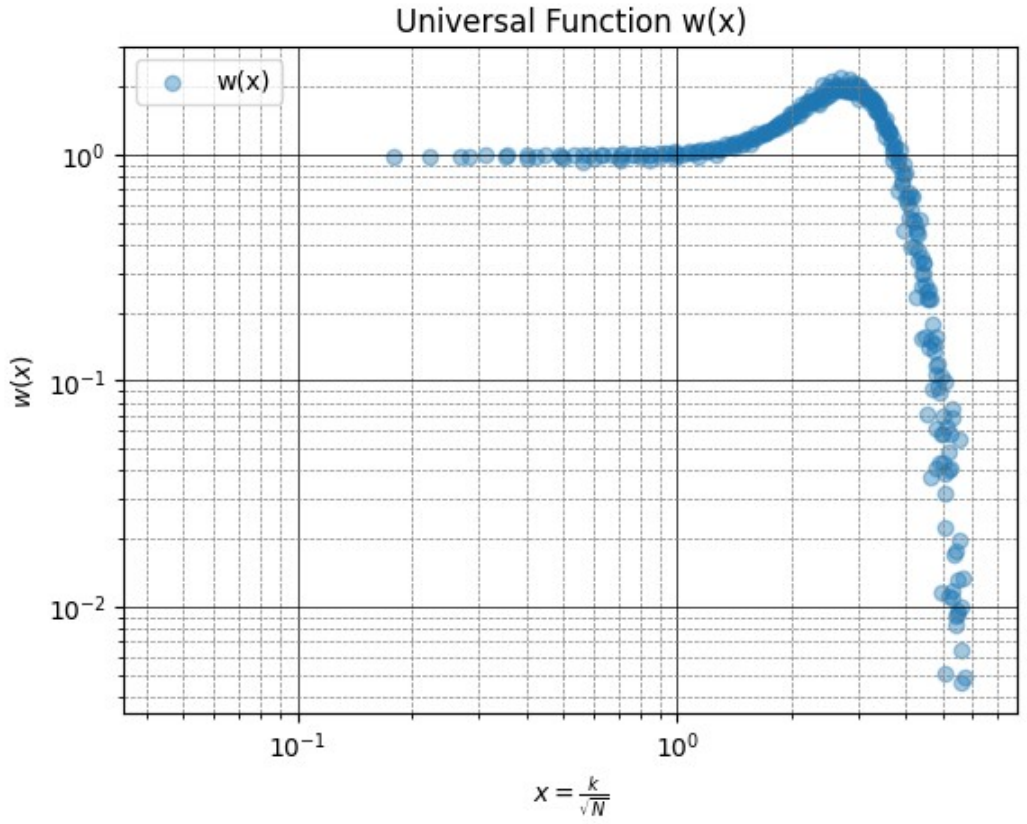

Computing $P_{est}/P_{\infty}$ to compare results:

In [96]:
```
# Plot the Finite Size Correction
plot_finite_size_correction(hist_avg, bins, N_vals, m, M)
plot_Wx(hist_avg, bins, N_vals, m, M)
```

```
C:\Users\Jarvis\AppData\Local\Temp\ipykernel_23856\3610188847.py:41: RuntimeWarning: invalid va
lue encountered in divide
  correction = hist_avg / P_inf_vals  # Finite size correction
```




```
C:\Users\Jarvis\AppData\Local\Temp\ipykernel_23856\3176151184.py:45: RuntimeWarning: invalid va
lue encountered in divide
  correction = hist_avg / P_inf_vals  # Finite size correction
```



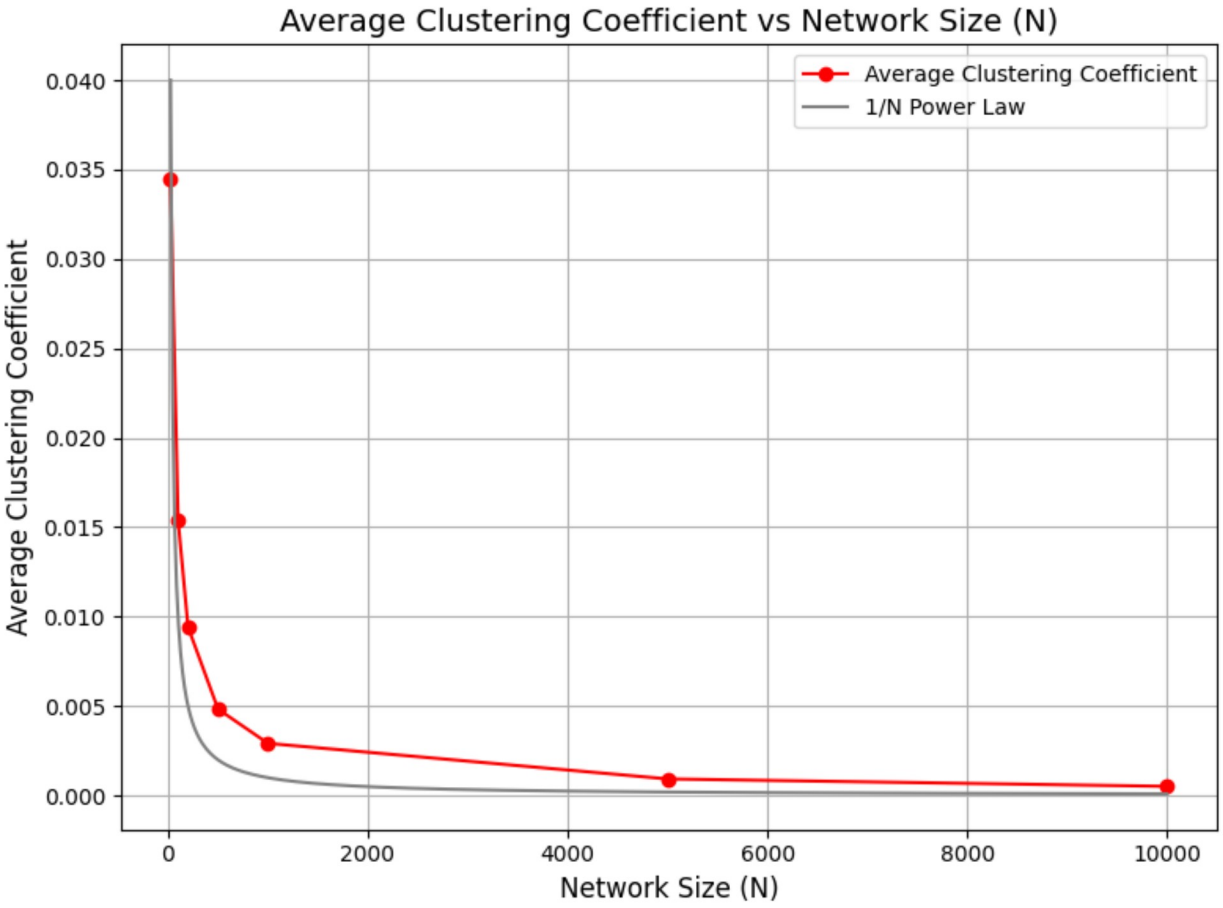

From this graph, we can see the numerical computations for the function of $w(x)$ plotted as a function of $x = \frac{k}{\sqrt{N}}$. This shows us the shape of the univeersal function for the finite size correction in eq.(4). This

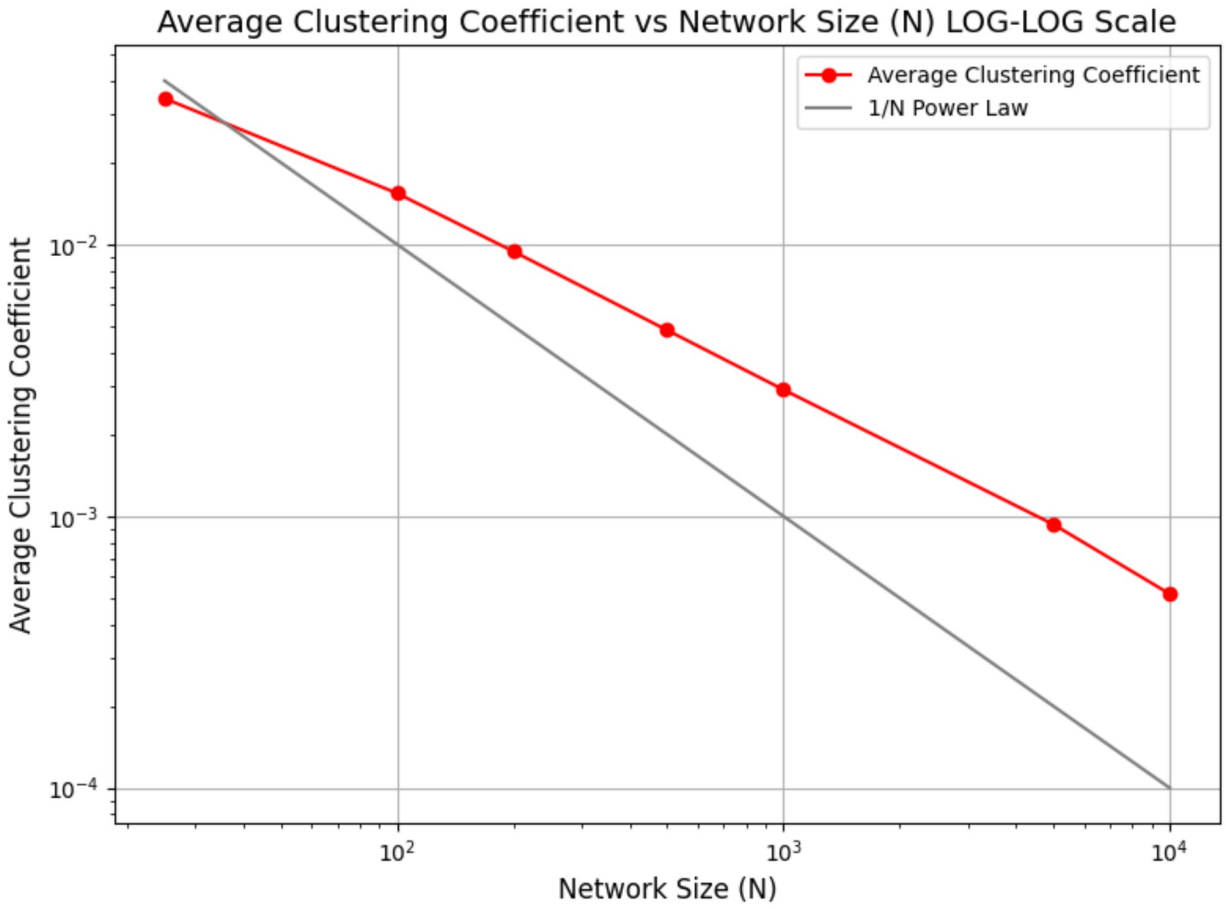
rough shape can also be seen in the previous graph of the finite size corrections across values of N.

## Part 4: Extra Point

*For a minimum degree m = 4, compute the average clustering coefficient 〈c〉 N for BA networks with size N = 25, 100, 200, 500, 1000, 5000 and 10000. Perform averages over M = 25 different networks. Plot the average clustering coefficient as a function of the network size*

In [98]:
```
# Should take around 40s
m = 4
N_vals = [25, 100, 200, 500, 1000, 5000, 10000]
M = 25

compute_avg_cluster_coef(N_vals,M,m)
```






Out[98]:
```
[0.034443032000679066,
 0.015418462474976835,
 0.009422843155082935,
 0.004835026186169569,
 0.002920501653473234,
 0.0009327443531190215,
 0.0005176110508156942]
```

We can see here that it seems to follow relatively closeley to a $N^{-1}$ power law. This is consistent for the average of the Cluster Coefficient for BA Networks, as is to be expected.